

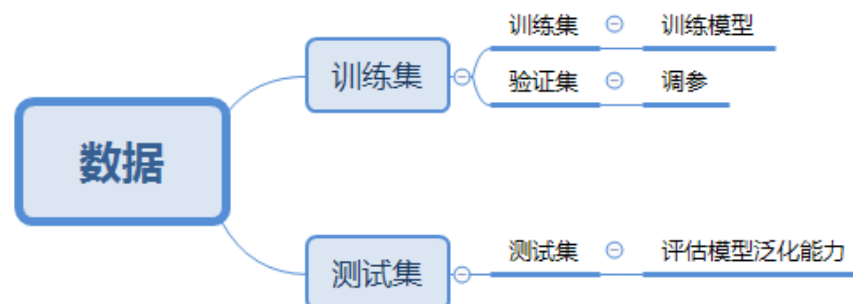


• 欠拟合与过拟合

■ 数据集

机器学习的普遍任务就是从数据中学习和构建模型（该过程称之为训练），并且能够在将来遇到的数据上进行预测

- **训练集 (training dataset)**：模型在训练上进行拟合，在对于监督式学习中是用来拟合参数的样本集合，输出量被称为目标或标签
- **验证集 (validation dataset)**：拟合模型在验证集上进行预测，在模型超参数调整时，提供对在训练集上拟合得到模型的无偏评估
验证集可用于正则化中的提前停止
- **测试集 (test dataset)**：提供对最终模型的无偏评估





• 欠拟合与过拟合

■ 泛化（Generalization）能力的定义

在机器学习中是指算法对**新鲜样本**的适应能力，即对具有同一规律的学习集以外的数据，经过训练的模型也能给出合适的输出

- 机器学习的基本问题是利用模型对数据进行拟合，学习的目的并非是对**有限训练集**进行正确预测，而是对未曾在训练集合（测试集合）出现的样本能够正确预测
- 模型对训练集数据的误差称为**经验误差**，对测试集数据的误差称为**泛化误差**
- 模型对**训练集以外**样本的预测能力就称为模型的**泛化能力**，追求这种泛化能力始终是机器学习的目标
- **过拟合**和**欠拟合**是导致模型泛化能力不高的两种常见原因，都是模型学习能力与数据复杂度之间失配的结果



• 欠拟合与过拟合

■ 泛化（Generalization）能力的定义

- “欠拟合”常常在模型**学习能力较弱**，而**数据复杂度较高**的情况出现，此时模型由于学习能力不足，无法学习到数据集中的“一般规律”，因而导致泛化能力弱
- “过拟合”常常出现在**模型学习能力过强**的情况，此时的模型**学习能力太强**，以至于将训练集**单个样本自身的特点**都能捕捉到，并将其认为是“**一般规律**”，同样这种情况也会导致模型泛化能力下降
- 过拟合与欠拟合的区别：欠拟合在训练集和测试集上的性能都较差，而过拟合往往能较好地学习训练集数据的性质，而在测试集上的性能较差
- 在神经网络训练的过程中，**欠拟合**主要表现为输出结果的**高偏差**，而**过拟合**主要表现为输出结果的**高方差**



• 欠拟合与过拟合

■ 偏差-方差分解（Bias-variance decomposition）的定义

从偏差和方差的角度来解释学习算法泛化性能的一种重要工具

符号	涵义
$\mathbf{x}$	测试样本
D	数据集
y_D	$\mathbf{x}$ 在数据集中的标记
y	$\mathbf{x}$ 的真实标记
f	训练集 D 学得模型
$f(\mathbf{x}; D)$	由训练集 D 学得模型 f 对 $\mathbf{x}$ 的预测输出
$\bar{f}(\mathbf{x})$	模型 f 对 $\mathbf{x}$ 的期望预测输出



欠拟合与过拟合

■ 偏差-方差分解（Bias-variance decomposition）的定义

从偏差和方差的角度来解释学习算法泛化性能的一种重要工具

- 偏差：用所有可能的训练数据集训练出的所有模型的输出的平均值与真实模型的输出值之间的差异

$$\text{bias}^2(\mathbf{x}) = (\bar{f}(\mathbf{x}) - y)^2$$

度量学习算法的期望预测与真实结果的偏离程度，即刻画了学习算法本身的拟合能力

- 方差：不同的训练数据集训练出的模型输出值之间的差异

$$\text{var}(\mathbf{x}) = \mathbb{E}_D[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2]$$

度量同样大小的训练集的变动所导致的学习性能的变化，即刻画了数据扰动所造成的影响

符号	含义
$\mathbf{x}$	测试样本
D	数据集
y	$\mathbf{x}$ 在数据集中的标记
$\bar{y}$	$\mathbf{x}$ 的真实标记
f	训练集 D 学到的模型
$f(\mathbf{x}; D)$	由训练集 D 学到的模型 f 对 $\mathbf{x}$ 的预测输出
$f(\mathbf{x})$	模型 f 对 $\mathbf{x}$ 的期望输出 $\mathbb{E}_D[f(\mathbf{x}; D)]$



• 欠拟合与过拟合

■ 偏差-方差分解（Bias-variance decomposition）的定义

- 噪声：数据必带有噪声，数据质量决定学习的上限

$$\varepsilon^2(\mathbf{x}) = \mathbb{E}_D[(y_D - y)]^2$$

表达在当前任务上任何学习算法所能达到的期望泛化误差的下界，即刻画了学习问题本身的难度

- 泛化误差为偏差、方差与噪声之和，即

$$\text{泛化误差 } \mathbb{E}(f; D) = \text{错误率}(\text{error}) = \text{bias}^2(\mathbf{x}) + \text{var}(\mathbf{x}) + \varepsilon^2(\mathbf{x})$$

因此，泛化性能是由**学习算法的能力**、**数据的充分性**以及**学习任务本身的难度**所共同决定的。给定学习任务，为了取得好的泛化性能，则需使偏差较小，即能够充分拟合数据，并且使方差较小，即使得数据扰动产生的影响小

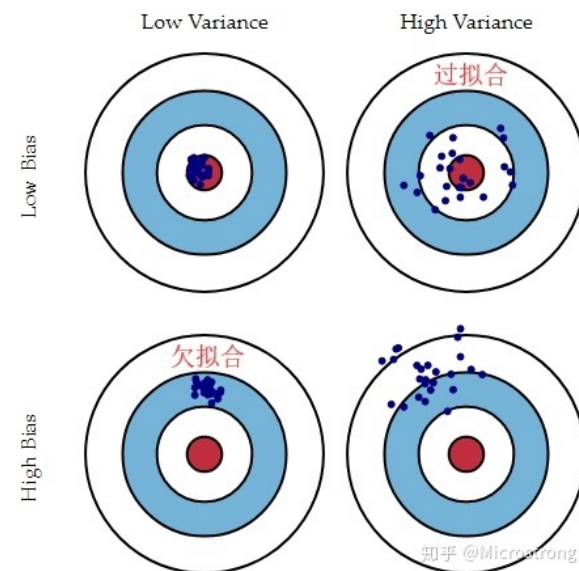


欠拟合与过拟合

■ 偏差和方差的性质

假设红色的靶心区域是学习算法完美的正确预测值，蓝色点为训练数据集所训练出的模型对样本的预测值，当从靶心逐渐往外移动时，预测效果逐渐变差

- 偏差的对象是单个模型，是期望输出与真实标记的差别，描述模型对本训练集的拟合程度
- 方差的对象是多个模型，是相同分布的不同数据集训练出模型的输出值之间的差异，刻画数据扰动对模型的影响
- 简单的模型会有一个较大的偏差和较小的方差（欠拟合），复杂的模型偏差较小方差较大（过拟合）



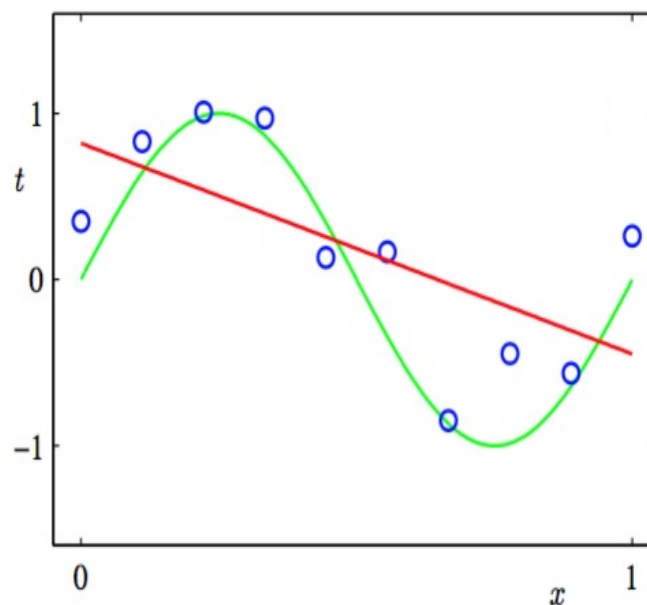
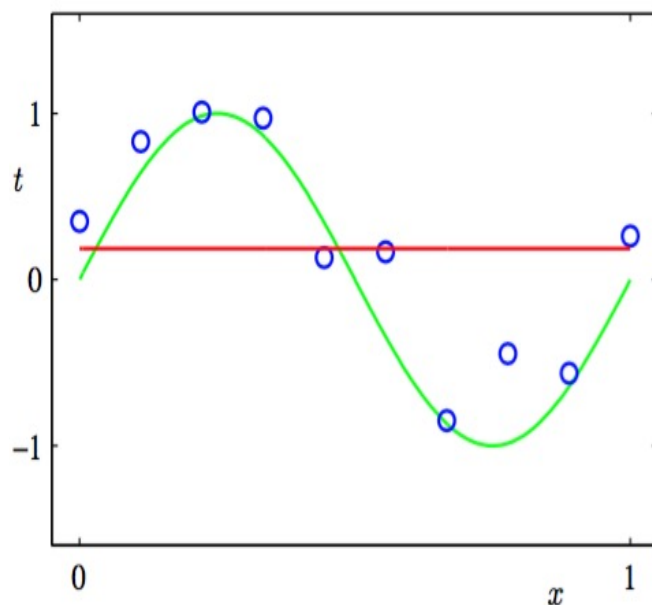


• 欠拟合与过拟合

■ 欠拟合（Under-fitting）的定义

欠拟合是指模型拟合程度不高，数据距离拟合曲线较远，或指模型没有很好地捕捉到数据特征，不能够很好地拟合数据

➢ 模型不能适配训练样本，有一个很大的偏差





• 欠拟合与过拟合

➤ 欠拟合的原因

一般是训练样本被提取的特征比较少，导致训练出来的模型不能很好地匹配，表现得很差，甚至样本本身都无法高效的识别

■ 欠拟合的解决方法

- **添加其他特征项**：有时候我们模型出现欠拟合的时候是因为特征项不够导致的，可以添加其他特征项来很好地解决；例如，“组合”、“泛化”、“相关性”三类特征，以及“上下文特征”、“平台特征”等等，都可作为特征添加的首选项
- **添加多项式特征**：在机器学习算法里面用的很普遍，例如将线性模型通过添加二次项或者三次项使模型泛化能力更强
- **减少正则化参数**：正则化的目的是用来防止过拟合的，但是现在模型出现了欠拟合，则需要减少正则化参数

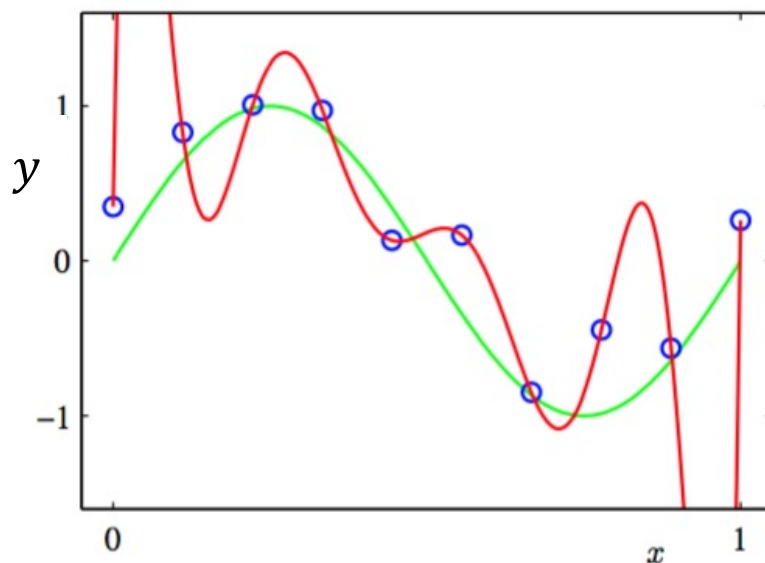


• 欠拟合与过拟合

■ 过拟合（Overfitting）的定义

在机器学习中，是测试数据所建的学习模型在训练样本中表现得过于优越，导致在验证数据集以及集中表现不佳

- 模型很好适配训练样本，但在测试集上表现差，有一个很大的方差
- 通常采用增大数据量和测试样本集的方法对分类器性能进行评价





• 欠拟合与过拟合

■ 过拟合（Overfitting）的定义

➢ 过拟合的原因

- **建模样本选取有误**，如样本数量太少，选样方法错误，样本标签错误等，导致选取的样本数据不足以代表预定的分类规则
- **样本噪音干扰过大**，使得机器将部分噪音认为是特征从而扰乱了预设的分类规则
- **模型的假设不合理**，或者说是假设成立的条件实际并不成立
- **参数太多**，模型复杂度过高
- 对于神经网络模型
 - ◆ 对样本数据可能存在分类决策面不唯一，随着学习的进行，反向传播算法使权值可能收敛过于复杂的决策面
 - ◆ 权值学习迭代次数足够多，拟合了训练数据中的噪声和训练样例中没有代表性的特征



• 欠拟合与过拟合

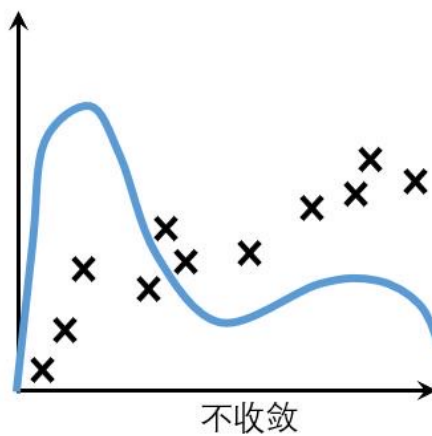
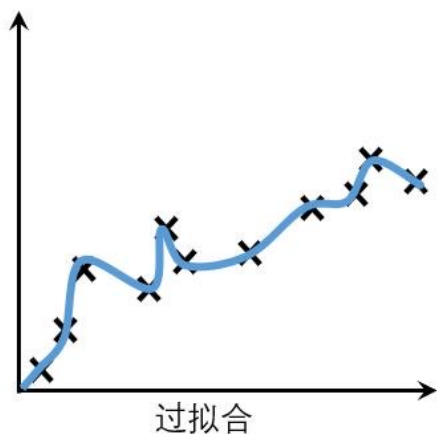
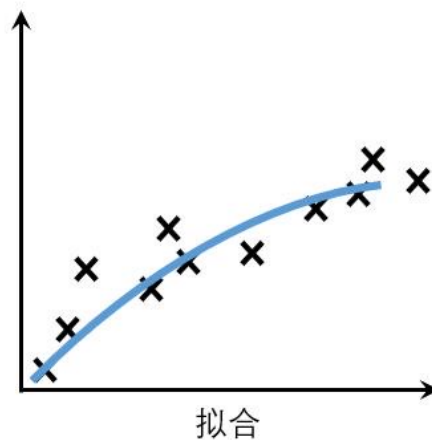
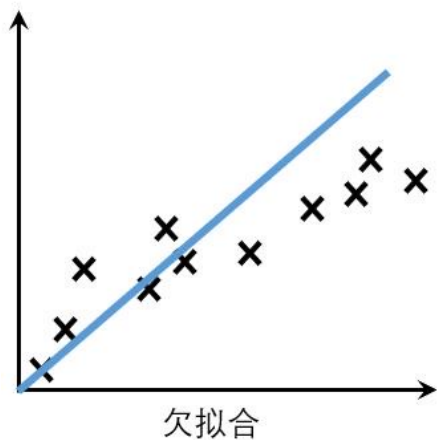
■ 过拟合的解决方法

- 重新清洗数据：导致过拟合的一个原因有可能是数据不纯导致的，因此如果出现了过拟合就需要我们重新清洗数据
- 增大数据的训练量：也可能是用于训练的数据量太小导致的，即训练数据占总数据的比例过小，因此增大数据的训练量
- 采用正则化方法：正则化方法包括 $L0$ 正则、 $L1$ 正则和 $L2$ 正则，而正则一般是在目标函数之后加上范数，在机器学习中一般使用 $L2$ 正则
- 采用Dropout丢弃法：在神经网络模型训练的过程中，对所有神经元按照一定的概率进行消除的处理方式
 - 在训练深度神经网络模型时，丢弃法能够在很大程度上简化神经网络结构，以达到防治过拟合的问题
 - 且能解决深度神经网络模型花费大量时间的问题



• 欠拟合与过拟合

■ 图示范例





• 正则化

■ 正则化（Regularization）的定义

通过向目标函数（也称经验风险）添加一个参数范数惩罚项来降低模型的容量，正则化后的损失函数变为

$$\tilde{L}(c; x, y) = L(c; x, y) + \alpha p(c)$$

其中 $\alpha \in [0, +\infty)$ 权衡范数惩罚项的相对贡献，值越大正则化程度越高

■ 正则化的常用方法

常见的正则项有 $L0$ 正则， $L1$ 正则和 $L2$ 正则

➢ $L0$ 参数正则化

$$\tilde{L}(c; x, y) = L(c; x, y) + \alpha ||c||_0$$

$L0$ 正则化是
NP难问题

- 使参数的元素大部分都为零，适用于机器学习中的稀疏编码和特征选择中，通过最小化 $L0$ 范数来寻找最少最优的稀疏特征项



过拟合与正则化

• 正则化方法

➤ L1参数正则化

$$\tilde{L}(c; x, y) = L(c; x, y) + \alpha \|c\|_1$$

其中，目标函数为凸函数，但非处处可微

- 亦称 LASSO (Least Absolute Shrinkage and Selection Operator, 稀疏规则算子) 规则化，学习地去掉这些没有信息的特征，即把这些特征对应的权重置为0，实现特征自动选择

➤ L2参数正则化

$$\tilde{L}(c; x, y) = L(c; x, y) + \alpha \|c\|_2$$

其中，目标函数为凸函数，处处可微

- L2 范数的规则项最小，可以使得向量的每个元素都很小并都接近于0，但不会让它等于0，越小的参数说明模型越简单

L2正则化还可以减小模型的Lipchitz常数，使得神经网络对输入扰动更加稳健



• 正则化方法

■ 正则化的常用方法

• $L2$ 规则化具体作用包括：

① 从学习理论的角度 $L2$ 可以防止过拟合

通过 $L2$ 范数，可以实现对模型空间的限制，使模型更简单，越简单的模型越不容易产生过拟合的现象，从而有效提升模型的泛化能力

② 从优化或者数值计算的角度 $L2$ 范数有助于病态矩阵求逆

$L2$ 范数有助于处理条件数不好的情况下矩阵求逆很困难的问题，使优化求解变得稳定和快速

■ $L0$ -范数与 $L1$ -范数的联系与区别

都可以实现稀疏，但是



• 正则化方法

■ L_0 -范数与 L_1 -范数的联系与区别

- L_0 -范数正则化本身是特征选择的最直接最理想的方案，但其不可解耦，且优化求解较难，
- L_1 -范数正则化比 L_0 -范数正则化具有更好的优化求解特性，因此实际应用中我们使用 L_1 -范数来得到 L_0 -范数的**最优凸近似**

■ L_1 -范数与 L_2 -范数的联系与区别

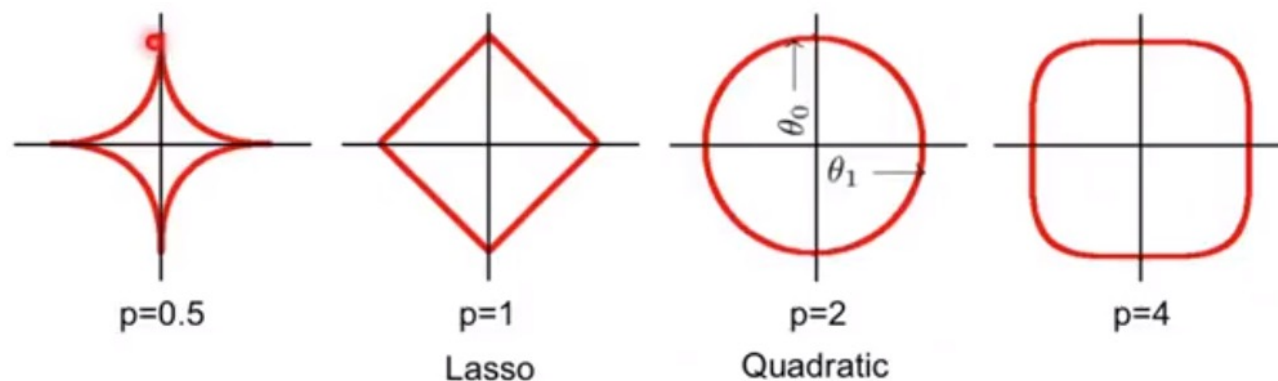
- 在最小化速度上：将权值参数以 L_1 -范数或者 L_2 -范数正则化的方式放到代价函数中，使模型尝试去最小化这些权值参数，**当在0附近时， L_1 -正则化的最小化速度比 L_2 -范数的最小化速度要快**
- 从最优化问题解上： L_1 -范数正则化的最优解数量少且多为全局最优解，而 L_2 -范数正则化的解很多，但其更多的倾向于某种局部最优解



• 正则化方法

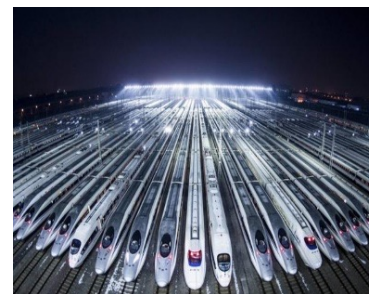
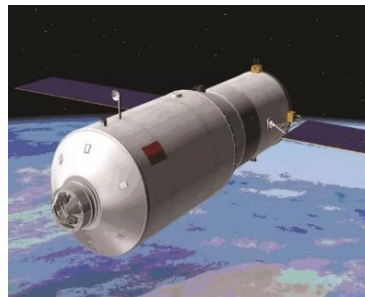
■ $L1$ -范数与 $L2$ -范数的联系与区别

- 从对模型空间的限制上： $L1$ -范数正则化会趋向于产生少量的特征，而其他的特征都是 0，而 $L2$ -范数正则化会选择更多的特征，这些特征都会接近于 0
- 从概率分析角度上：很多范数约束相当于对参数添加先验分布，其中 $L2$ 范数相当于参数服从**高斯先验分布**， $L1$ 范数相当于**拉普拉斯分布**



4-3

函数逼近





• 函数逼近

■ 函数逼近 (Approximation) 问题

用简单的函数 $g(x)$ 近似代替复杂的原函数 $f(x)$ ，且用 $g(x)$ 近似表示 $f(x)$ 所产生的误差可以量化，即对于 $f(x) \in C[a, b]$ ，在选定的函数类 $\Phi_n = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_n\} \subset C[a, b]$ 中寻找函数

$$g(x) \in \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_n\}$$

使 $g(x)$ 与函数 $f(x)$ 在一定准则下的近似，其中 $\varphi_i \in C[a, b] (i = 0, 1, \dots, n)$ ，且元素 $\varphi_0, \varphi_1, \dots, \varphi_n$ 线性无关

- 注意：逼近函数 $g(x)$ 具有不同的形式，包括最佳多项式逼近 (Optimal Polynomials)，切比雪夫近似 (Chebyshev Approximation)，雷米兹演算法 (Remez's Algorithm)
- 相同形式的逼近函数 $g(x)$ 具有多种确定方式，且 $g(x)$ 对 $f(x)$ 的近似程度或误差有不同定义，所以函数逼近问题的提法具有多种形式



• 函数逼近

■ 最佳逼近函数的定义

设 $\Phi_n \subset C[a, b]$ 为某个函数空间，给定函 $f(x) \in C[a, b]$ ，若存 $g^*(x) \in \Phi_n$ ，使得

$$\|f(x) - g^*(x)\|_p = \min_{g(x) \in \Phi_n} \|f(x) - g(x)\|_p$$

则称 $g^*(x)$ 为在区间 $[a, b]$ 上 $f(x)$ 在 Φ_n 中的**最佳逼近函数**

➤ 注意： $g^*(x)$ 与函数空间 Φ_n 、范数 $\|\cdot\|$ 和区间 $[a, b]$ 有关

- 若使用的范数为 $\|\cdot\|_\infty$ ，则称 $g^*(x)$ 为在区间 $[a, b]$ 上 $f(x)$ 在 Φ_n 中的**最佳一致逼近函数**
- 若使用的范数为 $\|\cdot\|_2$ ，则称 $g^*(x)$ 为在区间 $[a, b]$ 上 $f(x)$ 在 Φ_n 中的**最佳平方逼近函数**



• 函数逼近

■ 函数的最佳平方逼近

设 $f(x) \in C[a, b]$ ，若存在 $g^*(x) \in \Phi_n = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_n\}$ ，使得

$$\|f(x) - g^*(x)\|_2^2 = \min_{g(x) \in \Phi_n} \|f(x) - g(x)\|_2^2$$

$$= \min_{g(x) \in \Phi_n} \int_a^b (f(x) - g(x))^2 dx$$

则称 $g^*(x)$ 为在区间 $[a, b]$ 上 $f(x)$ 在 Φ_n 中的最佳平方逼近函数

► **定理：** $f(x)$ 在 Φ_n 中的最佳平方逼近函数存在且唯一

■ 最佳平方逼近的解法

设 $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x)$ 是空间 Φ_n 的一组基，则对任意 $g(x) \in \Phi_n$ ， $g(x)$ 可表示为

$$g(x) = \sum_{i=0}^n c_i \varphi_i(x)$$



• 函数逼近

■ 最佳平方逼近的解法

最佳平方逼近函数的参数 $c_0^*, c_1^*, \dots, c_n^*$ 的值可求解 **正规方程**

$$\begin{bmatrix} \langle \varphi_0, \varphi_0 \rangle & \langle \varphi_0, \varphi_1 \rangle & \cdots & \langle \varphi_0, \varphi_n \rangle \\ \langle \varphi_1, \varphi_0 \rangle & \langle \varphi_1, \varphi_1 \rangle & \cdots & \langle \varphi_1, \varphi_n \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle \varphi_n, \varphi_0 \rangle & \langle \varphi_n, \varphi_1 \rangle & \cdots & \langle \varphi_n, \varphi_n \rangle \end{bmatrix} \begin{bmatrix} c_0^* \\ c_1^* \\ c_2^* \\ c_3^* \end{bmatrix} = \begin{bmatrix} \langle f, \varphi_0 \rangle \\ \langle f, \varphi_1 \rangle \\ \vdots \\ \langle f, \varphi_n \rangle \end{bmatrix}$$

解得，从而确定

$$g^*(x) = \sum_{i=0}^n c_i^* \varphi_i(x)$$

记 $r(x) = f(x) - g^*(x)$ 为平方逼近误差，则有余项表达式

$$\|r(x)\|_2^2 = \|f(x) - g^*(x)\|_2^2 = \|f(x)\|_2^2 - \sum_{i=0}^n c_i^* \langle f, \varphi_i \rangle = \int_a^b f^2(x) dx - \sum_{i=0}^n c_i^* \int_a^b f(x) \varphi_i(x) dx$$



• 函数逼近

■ 最佳平方逼近多项式

已知 $\mathcal{P}_n$ 为区间 $[a, b]$ 上所有次数不超过 n 的多项式所组成的函数集合, 即 $\mathcal{P}_n = \text{span}\{1, x, \dots, x^n\} \subset C[a, b]$, 给定函数 $f(x) \in C[a, b]$, 若在 $\mathcal{P}_n$ 内存在元素

$$p_n(x) = c_0 + c_1x + \dots + c_nx^n \in \mathcal{P}_n$$

使得

$$\|f(x) - p^*(x)\|_2^2 = \min_{p(x) \in \mathcal{P}_n} \|f(x) - p(x)\|_2^2$$

则称 $p^*(x)$ 为在区间 $[a, b]$ 上 $f(x)$ 在 Φ 中的 **n 次最佳平方逼近多项式**

■ 最佳平方逼近多项式的解法

对于 $f(x) \in C[0, 1]$, $\mathcal{P}_n = \text{span}\{1, x, \dots, x^n\} \subset C[0, 1]$, 则

$$p_n(x) = \sum_i^n c_i x^i$$



• 函数逼近

■ 最佳平方逼近多项式的解法

$$\langle \varphi_i, \varphi_j \rangle = \int_0^1 x^{i+j} dx = \frac{1}{i+j+1}, \quad \langle f, \varphi_i \rangle = \int_0^1 f(x)x^i dx = d_i$$

此时，正规方程的系统矩阵为

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n+1} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \cdots & \frac{1}{n+2} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \cdots & \frac{1}{n+3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n+1} & \frac{1}{n+2} & \frac{1}{n+3} & \cdots & \frac{1}{2n+1} \end{bmatrix}$$

是一个希尔伯特矩阵（系数都是单位分数的方块矩阵，条件数大）



• 函数逼近

■ 最佳平方逼近多项式的解法

- 对于一般区间 $[a, b]$ 上的函数 $f(x)$, 可做变换 $x = a + (b - a)t$, 化为 $f(a + (b - a)t)$, $t \in [0, 1]$, 再作逼近处理, 减少积分计算
- 范例: 在区间 $[0, 1]$ 对于 $f(x) = e^x$ 的一次和二次最佳平方逼近多项式, 首先解得

$$d_0 = \int_0^1 e^x dx = e - 1 \approx 1.71828$$

$$d_1 = \int_0^1 x e^x dx = 1$$

$$d_2 = \int_0^1 x^2 e^x dx = e - 2 \approx 0.71828$$



• 函数逼近

■ 最佳平方逼近多项式的解法

➤ 范例：

- 因此，一次最佳平方逼近多项式的正规方程为

$$\begin{bmatrix} 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{3} \end{bmatrix} \begin{bmatrix} c_0^* \\ c_1^* \end{bmatrix} = \begin{bmatrix} 1.71828 \\ 1 \end{bmatrix}$$

解得

$$c_0^* = 0.87313, \quad c_1^* = 1.69031$$

则一次最佳平方逼近多项式为

$$p_1(x) = 0.87313 + 1.69031x$$

$$\text{则余项 } ||r(x)||_2^2 = ||f||_2^2 - \sum_{j=0}^1 c_j d_j = 1.4627 - = 0.00394$$



• 函数逼近

■ 最佳平方逼近多项式的解法

➤ 范例：

- 类似地，二次最佳平方逼近多项式的正规方程为

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ 1 & \frac{1}{3} & \frac{1}{4} \\ 2 & \frac{1}{3} & \frac{1}{4} \\ 1 & \frac{1}{4} & \frac{1}{5} \\ 3 & \frac{1}{4} & \frac{1}{5} \end{bmatrix} \begin{bmatrix} c_0^* \\ c_1^* \\ c_2^* \end{bmatrix} = \begin{bmatrix} 1.71828 \\ 1 \\ 0.71828 \end{bmatrix}$$

解得

$$c_0^* = 1.0129, c_1^* = 0.8515, c_2^* = 0.8388$$

则二次最佳平方逼近多项式为

$$p_2(x) = 1.0129 + 0.8515x + 0.8388x^2$$



• 函数逼近

■ 最佳平方逼近多项式的解法

➤ 范例：

- 从而，二次最佳平方逼近多项式的余项为

$$\|r(x)\|_2^2 = \|f\|_2^2 - \sum_{j=0}^2 c_j d_j = 1.4627 - 0.0000278$$

最佳二次平方逼近多项式比一次平方逼近多项式具有更好的逼近效果

■ 函数的最佳一致逼近

设 $f(x) \in C[a, b]$ ，若存在 $g^*(x) \in \Phi_n = \text{span}\{\varphi_0, \varphi_1, \dots, \varphi_n\}$ ，使得

$$\|f(x) - g^*(x)\|_\infty = \min_{g(x) \in \Phi_n} \|f(x) - g(x)\|_\infty = \min_{g(x) \in \Phi_n} \max_{a \leq x \leq b} |f(x) - g(x)|$$

则称 $g^*(x)$ 为在区间 $[a, b]$ 上 $f(x)$ 在 Φ_n 中的最佳一致逼近函数



• 函数逼近

■ 维尔斯特拉斯 (Weierstrass) 第一定理

设 $f(x) \in C[a, b]$ ，则对于任意 $\varepsilon > 0$ ，存在一个多项式 $p_n \in \mathcal{P}_n$ ，使得

$$\|f(x) - p_n(x)\|_{\infty} := \max_{a \leq x \leq b} |f(x) - p(x)| < \varepsilon$$

在 $[a, b]$ 上**一致**成立

- 任意一个闭区间上的连续函数都可以用多项式来**一致逼近**，即实系数多项式构成的集合在 $C[a, b]$ 内是处处稠密
- 另外维尔斯特拉斯第二定理也证明了闭区间上周期为 2π 的连续函数可用三角函数级数逼近



• 函数逼近

■ 最佳一致逼近多项式的近似解法

近似的最佳一致逼近多项式 $p(x)$ 在区间 $[-1,1]$ 上截断切比雪夫级数部分和形式如下：

$$p(x) = \sum_{k=0}^n a_k T_k(x)$$

其中， $a_0 = \frac{1}{\pi} \int_0^{\pi} f(\cos \theta) d\theta$, $a_k = \frac{2}{\pi} \int_0^{\pi} f(\cos \theta) \cos(k\theta) d\theta$, $k = 1, 2, \dots$

$$T_k(x) = \cos(k \arccos x), x \in [-1, 1]$$

为切比雪夫多项式，满足 $|T_k(x)| \leq 1$

- 切比雪夫多项式可根据递推公式 $T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x)$ 计算出，一般多项式的形式为 $T_0(x) = 1$, $T_1(x) = x$, $T_2(x) = 2x^2 - 1$, $T_3(x) = 4x^3 - 3x, \dots$



• 函数逼近

■ 近似最佳一致逼近问题的求解

- 例如： $f(x) = e^x$ 在区间 $[-1,1]$ 上的三次切比雪夫近似最佳一致逼近多项式可求得

$$a_0 = \frac{1}{\pi} \int_0^\pi f(\cos \theta) d\theta = \frac{1}{\pi} \int_0^\pi e^{\cos \theta} d\theta = 1.266066$$

$$a_1 = \frac{2}{\pi} \int_0^\pi f(\cos \theta) \cos \theta d\theta = \frac{2}{\pi} \int_0^\pi e^{\cos \theta} \cos \theta d\theta = 1.130318$$

$$a_2 = \frac{2}{\pi} \int_0^\pi f(\cos \theta) \cos 2\theta d\theta = \frac{2}{\pi} \int_0^\pi e^{\cos \theta} \cos 2\theta d\theta = 0.271495$$

$$a_3 = \frac{2}{\pi} \int_0^\pi f(\cos \theta) \cos 3\theta d\theta = \frac{2}{\pi} \int_0^\pi e^{\cos \theta} \cos 3\theta d\theta = 0.044337$$

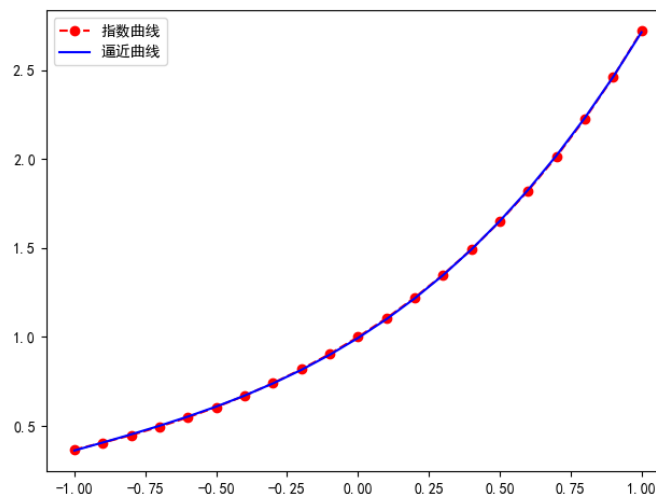


• 函数逼近

■ 近似最佳一致逼近问题的求解

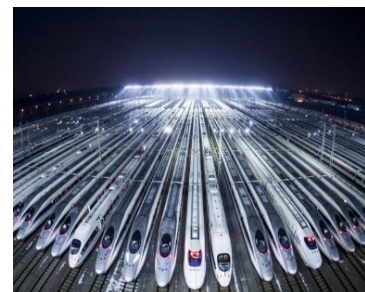
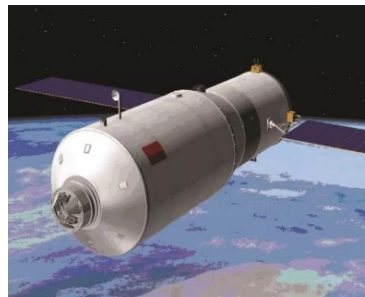
➤ 所以

$$\begin{aligned} p(x) &= a_0 T_0(x) + a_1 T_1(x) + a_2 T_2(x) + a_3 T_3(x) \\ &= 1.266066 + 1.130318x + 0.271495(2x^2 - 1) + 0.044337(4x^3 - 3x) \\ &= 0.177347x^3 + 0.542991x^2 + 0.997308x + 0.994571 \end{aligned}$$



4-4

核函数及核函数逼近





• 超平面

■ 超平面（Hyperplane）的基本定义

是 n 维欧氏空间中余维度等于一的线性子空间，也就是超平面是 n 维空间中的 $n - 1$ 维的子空间，是平面中的直线、空间中的平面的推广

➤ 超平面的本质是自由度比所在空间的维度小 1

➤ 范例：

- 一维空间中（超）平面为数轴上的一个点，自由度为 0
- 二维空间中超平面只要给定 (x, y) 中任意一个分量，就可以确定剩下一个分量的值，即二维空间中超平面为一条直线，自由度为 1
- 三维空间中（超）平面为二维平面，自由度为 2
- ...
- n 维空间中（超）平面为 $n - 1$ 维平面，自由度为 $n - 1$



• 超平面

■ 超平面的表示方法

n 维空间中超平面方程表示为

$$\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$$

- 列向量 $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ ，即可以看作超平面的法向量，也可以看作是参数，决定超平面的方向
- 列向量 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ 为超平面上的点
- 实数 b 表示超平面到原点的距离
- 符号 $\langle \cdot, \cdot \rangle$ 表示向量内积，在欧式空间中，有 $\langle \mathbf{w}, \mathbf{x} \rangle = \mathbf{w}^T \mathbf{x}$
- 超平面将空间划分为3部分，即
 - 超平面本身： $\{\mathbf{x} \in \mathbb{R}^n \mid \langle \mathbf{w}, \mathbf{x} \rangle + b = 0\}$
 - 超平面上部（正面）： $\{\mathbf{x} \in \mathbb{R}^n \mid \langle \mathbf{w}, \mathbf{x} \rangle + b > 0\}$
 - 超平面下部（反面）： $\{\mathbf{x} \in \mathbb{R}^n \mid \langle \mathbf{w}, \mathbf{x} \rangle + b < 0\}$



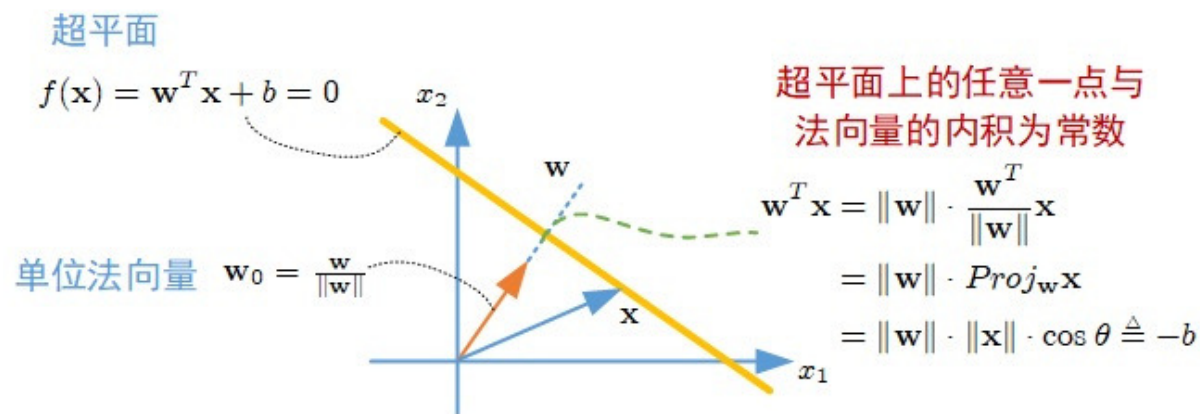
• 超平面

■ 超平面的表示方法

- 在机器学习中，需要先利用训练数据学习确定超平面方程中的参数 w 和 b ，之后将测试数据 x 代入 $\langle w, x \rangle + b$ 得到值，依据

$$x \text{ 在超平面的} = \begin{cases} \text{正面,} & \langle w, x \rangle + b > 0 \\ \text{平面上,} & \langle w, x \rangle + b = 0 \\ \text{反面,} & \langle w, x \rangle + b < 0 \end{cases}$$

法向量指向的那一半对应的一面是它的正面，另一面则是它的反面





核函数原理

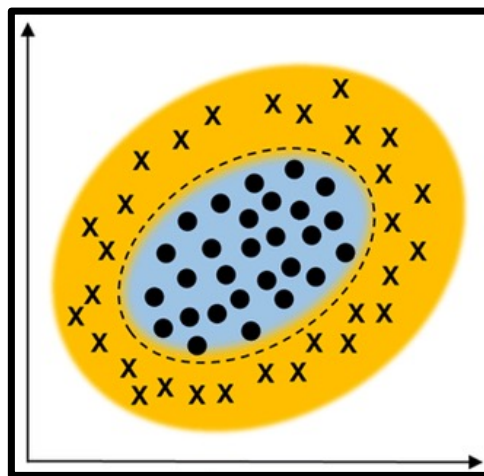
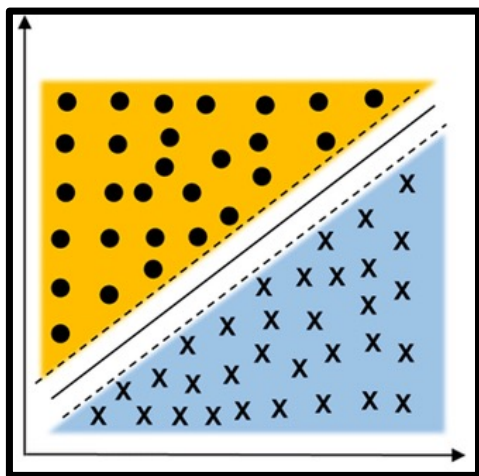
• 线性分类

■ 线性可分的基本定义

在欧几里德空间中，若用一个分类超平面可将不同数据完全划分，则这两组数据是线性可分的

■ 非线性可分的基本定义

在欧几里德空间中，若不存在一个分类超平面可将不同数据完全划分，则这两组数据是非线性可分的

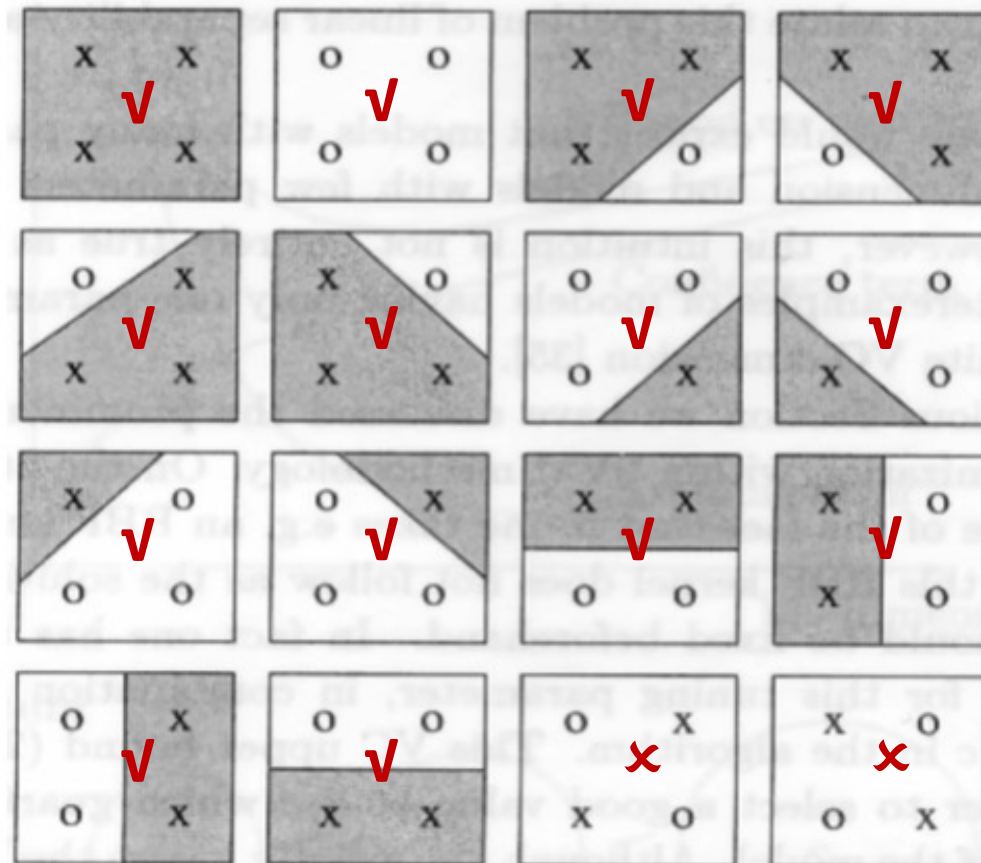




核函数原理

• 线性分类

■ 线性分类器极限



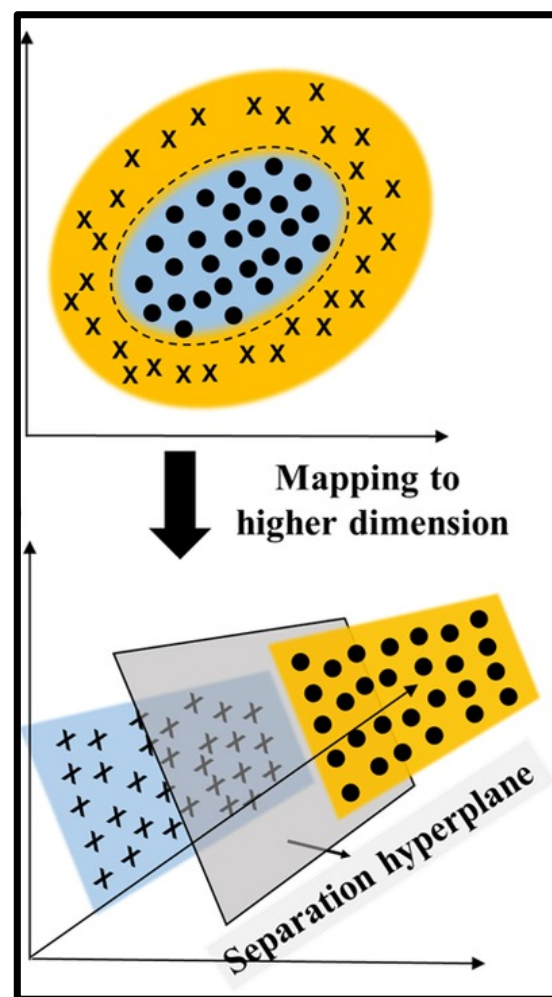


• 升维

机器学习中，样本在低空间是非线性可分的，而在高维空间是线性可分的，通过数据的**空间变换**，将原低维输入空间的样本数据通过**映射函数**转变为高维（也可以是无穷维）特征空间的数据，进而实现线性可分

■ 升维的基本定义

就是把数据从原输入低维空间向高维特征空间作映射，使得数据的维度增大，从而解决低维空间非线性可分的问题



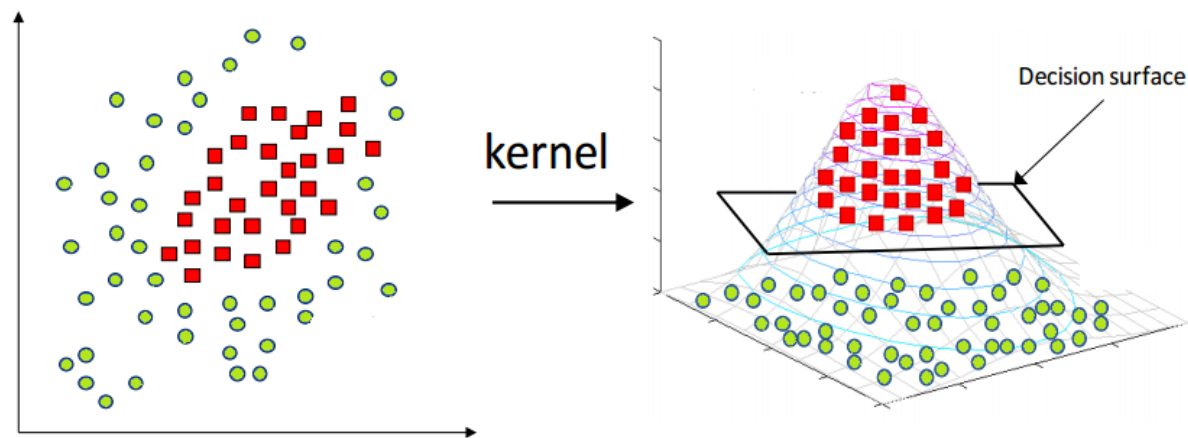


• 升维

■ 升维面临挑战

机器学习中，包括支持向量机（SVM）、K近邻算法（KNN）、线性回归和聚类等，需要计算特征空间中数据之间的内积，当将低维数据映射到高维数据时，求解内积运算的计算量将变得大，而且计算非常困难

- 通过引入核函数实现在低维空间获取高维空间的计算结果，即避免了直接在高维空间中的复杂计算，又达到高维运算的效果





• 核函数

■ 核函数（Kernel Function）的定义

设 $\mathbb{X}$ 是输入空间， $\mathbb{Y}$ 为标签域， $\mathbb{F}$ 为特征空间，给定训练集

$$S = \{(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_n, \mathbf{y}_n)\}$$

其中测量 $\mathbf{x}_i \in \mathbb{X} \subset \mathbb{R}^n$ 对应标签 $\mathbf{y}_i \in \mathbb{Y} \subset \mathbb{R}^m$ ，如果非线性函数 $\phi(\mathbf{x})$ 可实现从输入空间 $\mathbb{X}$ 到特征空间 $\mathbb{F}$ 的映射，其中 $\mathbb{F} \subset \mathbb{R}^p$ ，即存在从 $\mathbb{X}$ 到某个特征空间 $\mathbb{F}$ 的映射，使得满足

$$k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$$

其中 $\langle \cdot, \cdot \rangle$ 为内积， $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ 表示 $\mathbf{x}_i, \mathbf{x}_j$ 映射到特征空间上的向量之间的内积， $k(\cdot, \cdot): \mathbb{X} \times \mathbb{X} \rightarrow \mathbb{R}$ 称为**核函数**， $\phi(\mathbf{x}_i)$ 称为**隐藏函数**

- 核函数的引入可以使内积作为输入空间的直接函数，更高效地计算内积，而不用显示工计算映射



• 核函数

■ 核函数（Kernel Function）的定义

- 范例：设 $\phi(\mathbf{x})$ 定义为将三维的数据 x_1, x_2, x_3 映射到九维的数据空间中，即

$$\phi(\mathbf{x}) = [x_1x_1, x_1x_2, x_1x_3, x_2x_1, x_2x_2, x_2x_3, x_3x_1, x_3x_2, x_3x_3]$$

现有两个样本 $\mathbf{x}_1 = [1, 2, 3], \mathbf{x}_2 = [4, 5, 6]$ ，那么

$$\phi(\mathbf{x}_1) = [1, 2, 3, 2, 4, 6, 3, 6, 9]$$

$$\phi(\mathbf{x}_2) = [16, 20, 24, 20, 25, 30, 24, 30, 36]$$

那么 $\langle \phi(\mathbf{x}_1), \phi(\mathbf{x}_2) \rangle = 1024$

另一方面，可计算

$$k(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2)^2 = 1024$$

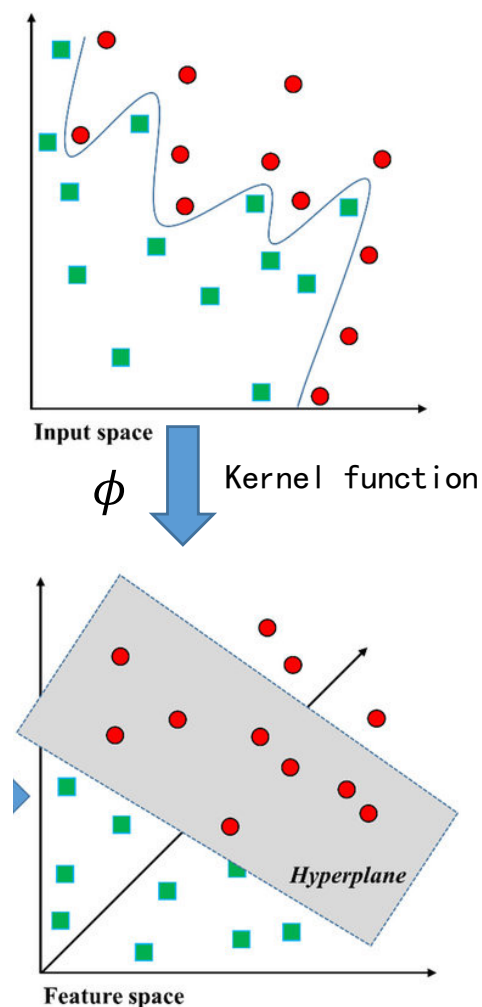
- 核函数的计算量要比映射函数小，维度越大，效率区分越大



• 核函数

■ 核函数的特点

- 核函数是在原空间进行计算，既避免了“维数灾难”，又大大减小了计算量，因此，核函数方法可以有效处理高维输入
- 无须知道变换函数 $\phi(x)$ 的形式和参数
- 核函数的形式和参数的变化会改变从输入空间到特征空间的映射，进而对特征空间的性质产生影响，最终改变核函数方法的性能
- 核函数方法可以和不同的算法相结合，形成多种不同的基于核函数计算的方法，且这两部分的设计可以单独进行，并可为不同的应用选择不同的核函数和算法





• 核函数

■ 默瑟 (Mercer) 定理

设 $\mathbb{X}$ 是输入空间, $k(\cdot, \cdot)$ 是定义在 $\mathbb{X} \times \mathbb{X}$ 上的对称核函数, 则 $k(\cdot, \cdot)$ 是核函数当且仅当对于任意数据 $\{x_1, x_2, \dots, x_m\}$, “核矩阵”

$$K = \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_j) & \cdots & k(x_1, x_m) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ k(x_i, x_1) & \cdots & k(x_i, x_j) & \cdots & k(x_i, x_m) \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ k(x_m, x_1) & \cdots & k(x_m, x_j) & \cdots & k(x_m, x_m) \end{bmatrix}$$

总是半正定的, 即如果对任意非零向量 $\mathbf{x}$, 半正定矩阵都有

$$\mathbf{x}^T K \mathbf{x} \geq 0$$

➤ 只要一个对称函数所对应的核矩阵半正定, 则可作为核函数使用



• 常见的核函数

■ 线性核函数

线性核函数主要用于线性可分，其定义为

$$k(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i, \mathbf{x}_j \rangle = \mathbf{x}_i^T \mathbf{x}_j$$

即映射函数为 $\phi(\mathbf{x}_i) = \mathbf{x}_i$

- 线性核函数的特征空间与输入空间的维度一样，对数据不作任何变换，不需要设置任何参数

■ 多项式核函数

线性核函数主要用于线性可分，其定义为

$$k(\mathbf{x}_i, \mathbf{x}_j) = [a \langle \mathbf{x}_i, \mathbf{x}_j \rangle + b]^c$$

其中

- $a > 0$ 表示对内积 $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ 进行缩放，一般等于 $\frac{1}{\text{类别数}}$



• 常见的核函数

■ 多项式核函数

- 常数项 $b \geq 0$ ，当 $b = 0$ 为齐次多项式，否则为非齐次多项式
- c 代表多项式的阶次，升维的维度随 c 的增大而指数倍增长，计算量也随之增大； c 取值过大，学习的复杂性也会过高，容易出现过拟合
- 范例：对于多项式核函数

$$\begin{aligned} k(\mathbf{x}_i, \mathbf{x}_j) &= [\mathbf{x}_i^T \mathbf{x}_j + 1]^2 = \left[\sum_{k=1}^n x_i^k x_j^k + 1 \right]^2 \\ &= \sum_{k=1}^n (x_i^k x_j^k)^2 + \sum_{k=2}^n \sum_{p=1}^{k-1} (\sqrt{2} x_i^k x_i^p) (\sqrt{2} x_j^k x_j^p) + \sum_{k=1}^n 2 x_i^k x_j^k + 1 \end{aligned}$$

$$\therefore \phi(\mathbf{z}) = [z_n^2, z_{n-1}^2, \dots, z_1^2, \sqrt{2} z_n z_{n-1}, \dots, \sqrt{2} z_2 z_1, \sqrt{2} z_n, \sqrt{2} z_{n-1}, \dots, \sqrt{2} z_1, 1]$$

可验证 $k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$



• 常见的核函数

■ 高斯径向基核函数

高斯径向基核函数是应用最为广泛的一种核函数，表达式为

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

其中核半径 $\sigma > 0$ 是自定义用于确定到达率或者说函数值跌落到 0 的速度参数

- 若 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 很相近，则核函数等于 1；若 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 相差很大，则核函数值约为 0
- 径向基函数是指数形式，展开是无穷次的多项式，因而径向基函数可以将原始特征数据映射到无穷维空间
- 高斯径向基核函数对于大样本和小样本均适用，相比于多项式核函数具有更少参数



• 常见的核函数

■ 高斯径向基核函数

➤ 范例：对于高斯径向基核函数

$$k(\mathbf{x}_i, \mathbf{x}_j) = e^{(-\|\mathbf{x}_i - \mathbf{x}_j\|^2)}$$

给定 n 维向量 $\mathbf{x}_i, \mathbf{x}_j$ ，因为

$$k(\mathbf{x}_i, \mathbf{x}_j) = e^{(-\|\mathbf{x}_i - \mathbf{x}_j\|^2)} = e^{-(\mathbf{x}_i^2 + \mathbf{x}_j^2 - 2\mathbf{x}_i\mathbf{x}_j)} = e^{-\mathbf{x}_i^2} e^{-\mathbf{x}_j^2} e^{2\mathbf{x}_i\mathbf{x}_j}$$

$$= e^{-\mathbf{x}_i^2} e^{-\mathbf{x}_j^2} \left[\sum_{k=1}^{\infty} \frac{(2\mathbf{x}_i\mathbf{x}_j)^k}{k!} \right] = e^{-\mathbf{x}_i^2} e^{-\mathbf{x}_j^2} \left[\sum_{k=1}^{\infty} \frac{(2\mathbf{x}_i\mathbf{x}_j)^k}{k!} \right]$$

$$= e^{-\mathbf{x}_i^2} e^{-\mathbf{x}_j^2} \left[1 + \frac{(2\mathbf{x}_i\mathbf{x}_j)^1}{1!} + \frac{(2\mathbf{x}_i\mathbf{x}_j)^2}{2!} + \frac{(2\mathbf{x}_i\mathbf{x}_j)^3}{3!} + \dots \right]$$

因为括号内每一项都是多项式核函数，所以高斯径向基核函数对应的映射函数映射之后是无穷维的



• 常见的核函数

■ 核函数的选择

在实际问题中选择核函数，采用如下的方法

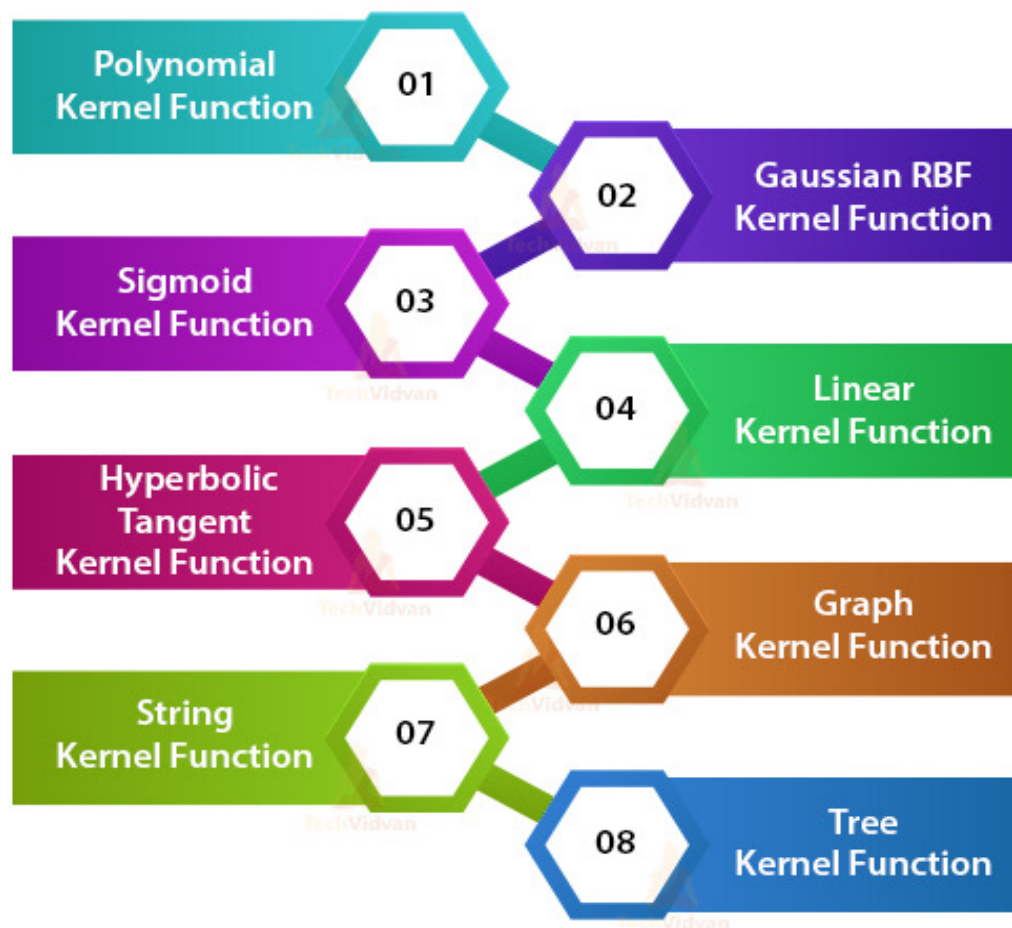
- 根据先验知识预先选定核函数，针对特征向量类型选用函数
 - **线性核函数**：主要用于线性可分的情形；参数少，速度快，对于一般数据，分类效果较理想，适用于高维度、样本数量差异性低的数据集，当维数较少时，样本数量较多，可以手动添加一些维数，再使用线性核函数
 - **高斯径向基核函数**：使用范围广，是支持向量机（SVM）默认的核函数，适用于维数较低和样本数量一般的数据集，主要用于非线性可分的情形
 - **多项式核函数**：适用于图像处理，可调节参数（可利用交叉验证或枚举法）获得较好的效果



• 常见的核函数

- 核函数的选择
 - 采用交叉验证方法，在选取核函数时，分别试用不同的核函数，通过仿真实验，在相同数据条件下对比分析，归纳误差最小的核函数
 - 采用混合核函数方法，即将不同的核函数结合起来使用

Types of Kernel Functions





• 支持向量机

■ 最优超平面的定义

对于满足

- **最近距离最远**：在两类样本中，每类都有距离超平面最近的样本，而“最优的”超平面是使这两类最近的样本到该超平台的距离（间隔）尽可以远

- **等距**：指离超平面最近的两类样本到超平面的距离是相等的
的超平面称为“最优超平面”

- 即它是在所有分类超平面中分类效果最好的
- 这个超平面能够准确地把样本分开且不存在错误分类
- 空间中最靠近分类超平面的点到超平面的距离是相等且最大的
- 支持向量机是一种**二分类**模型，它的基本模型是定义在特征空间上的间隔最大的线性分类器，间隔最大使它有别于感知机

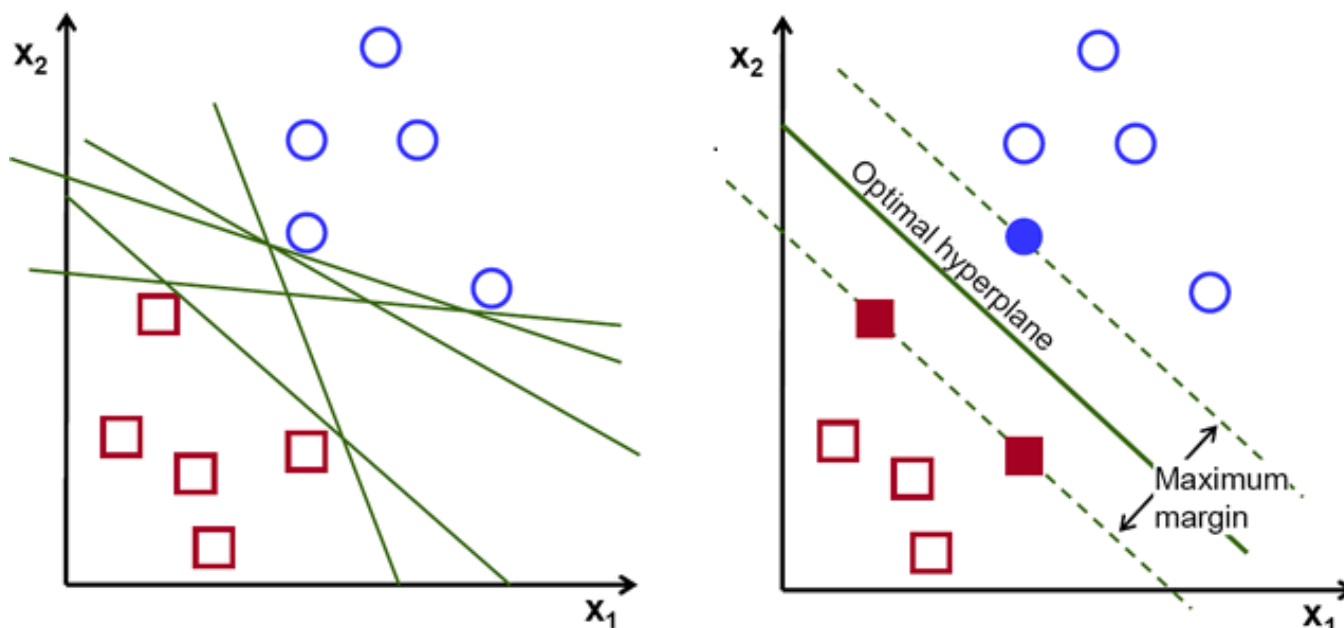


支持向量机原理

• 支持向量机

■ 支持向量机（Support Vector Machine, SVM）的定义

又名支持向量网络，两类样本中，离最优分类超平面最近的点且平行于最优分类超平面的训练样本（向量）称为支持向量，其“撑”起了分类超平面





• 支持向量机

- 支持向量机（Support Vector Machine, SVM）的定义
是一类按**监督学习**方式对数据进行二元分类的广义线性分类器，其决策边界是对学习样本求解的**最大边距超平面** (Maximum-margin Hyperplane)
- 支持向量机的分类
 - 线性可分支持向量机：当训练样本线性可分时，通过硬间隔最大化学习一个线性分类器，得到为直线或平面的超平面
 - 线性支持向量机：当训练数据近似线性可分时，引入松弛变量，通过软间隔最大化学习一个线性分类器
 - 非线性支持向量机：当训练数据是非线性可分时，通过使用核函数及软间隔最大化



• 支持向量机

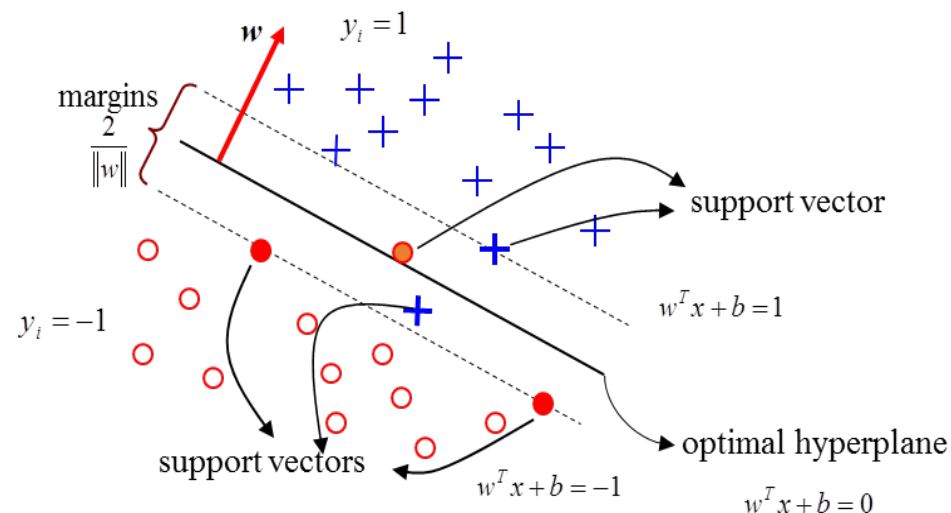
■ 线性可分支持向量机的定义

对于测试集 $\mathcal{S} = \{(x_1, y_1), \dots, (x_n, y_n)\}$, $x_i \in \mathbb{X} \subset \mathbb{R}^n$, $y_i \in \{-1, 1\}$ 表示 x_i 所属分类, 要求 $y_i = 1$ 与 $y_i = -1$ 的点集分开的“最大间隔超平面”, 使得超平面与最近的点 x_i 的距离最大化; 特别的, 任何超平面都可以写作满足下面方程的点集

$$\langle w, x \rangle + b = 0$$

其中 $w = [w_1, w_2, \dots, w_n]^T$ 为法向量, b 为偏置, 参

数 $\frac{b}{\|w\|}$ 决定从原点沿法向量 w 到超平面的偏移量





• 支持向量机

➤ 硬间隔

如果训练数据是线性可分的，可以选择分离两类数据的两个平行超平面，使得它们之间的距离尽可能大，最大“间隔”超平面是位于它们正中间的超平面

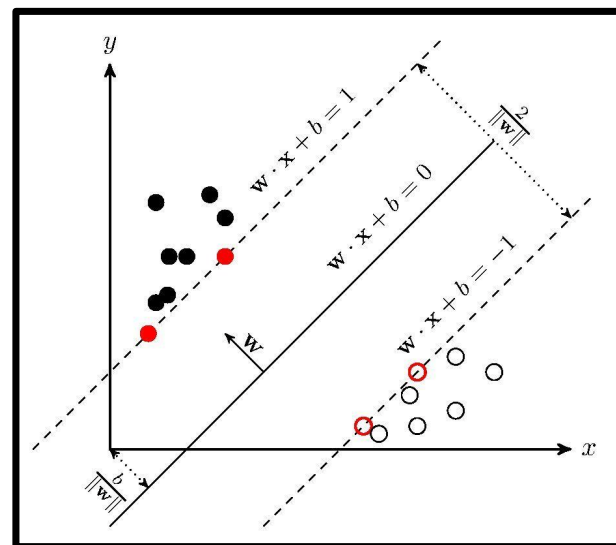
- 假设超平面 $(\mathbf{w}, b)$ 能将训练样本正确分类，即对于 $(\mathbf{x}_i, y_i) \in \mathcal{S}$

若 $y_i = +1$ ，则有 $\langle \mathbf{w}, \mathbf{x}_i \rangle + b > 0$ ；若

$y_i = -1$ ，则有 $\langle \mathbf{w}, \mathbf{x}_i \rangle + b < 0$ ，即

$$\begin{cases} \langle \mathbf{w}, \mathbf{x}_i \rangle + b \geq +1, & y_i = +1 \\ \langle \mathbf{w}, \mathbf{x}_i \rangle + b \leq -1, & y_i = -1 \end{cases}$$

这两个超平面之间的距离是 $\frac{2}{\|\mathbf{w}\|}$ ，因此要使两平面的距离最大，我们需要最小化 $\|\mathbf{w}\|$





支持向量机原理

• 支持向量机

➤ 硬间隔

换言之，两式合并后可写作

$$y_i(< \mathbf{w}, \mathbf{x}_i > + b) \geq 1, \quad \forall 1 \leq i \leq n$$

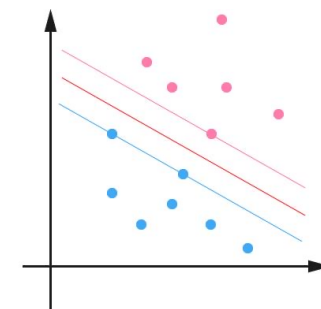
因此，最大间隔超平面可由解决下列优化问题得到

$$\begin{aligned} \min_{\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}} \quad & \|\mathbf{w}\| \\ \text{s.t.} \quad & y_i(< \mathbf{w}, \mathbf{x}_i > + b) \geq 1 \quad \forall 1 \leq i \leq n \end{aligned}$$

该问题的最优解 $\mathbf{w}^*, b^*$ 决定分类器

$$\mathbf{x} \mapsto \text{sgn}(< \mathbf{w}^*, \mathbf{x}_i > + b^*)$$

- 几何上，最大间隔超平面完全是由最靠近它的样本数据 $\mathbf{x}_i$ 确定，即支持向量





支持向量机原理

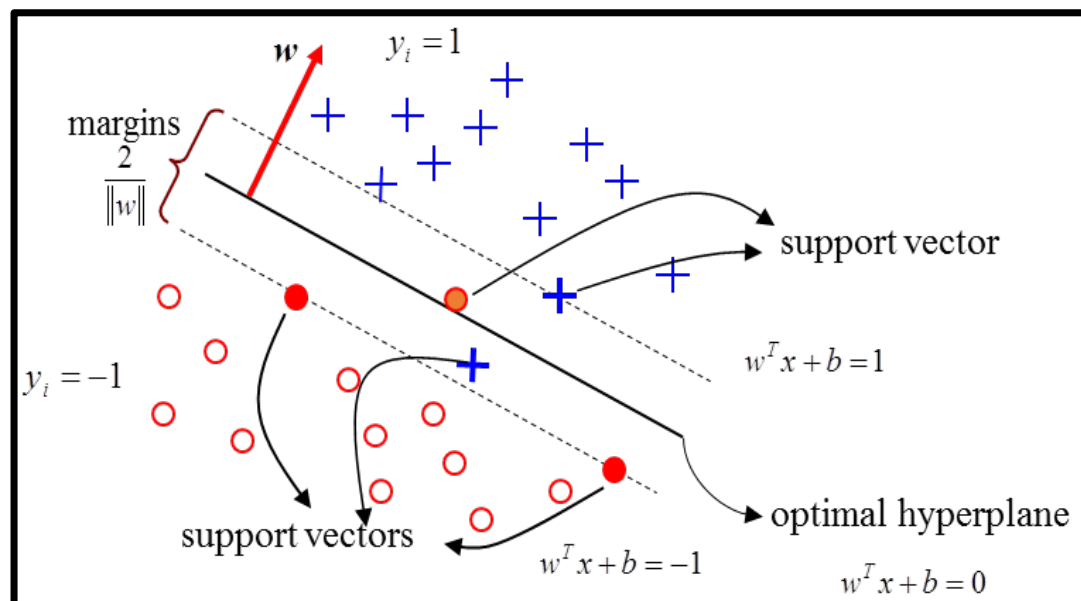
• 支持向量机

■ 线性支持向量机的

将支持向量机拓展到线性不可分的情况，引入铰链损失（Hinge loss，也译为合页损失）函数

$$\max(0, 1 - y_i(\langle w, x_i \rangle + b))$$

当约束条件 $y_i(\langle w, x_i \rangle + b) \geq 1$ （即如果 x_i 位于边距的正确一侧）满足时，此函数为零；对于间隔错误一侧的数据，该函数的值与离间隔的距离成正比





• 支持向量机

- 线性支持向量机的

- 软间隔

最终，解决正则化的合页损失函数的最小化问题

$$\min_{w,b} \left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\langle w, x_i \rangle + b)) \right] + \rho \|w\|^2$$
$$s.t. \ y_i(\langle w, x_i \rangle + b) \geq 1, \forall 1 \leq i \leq n$$

ρ 用来权衡增加间隔大小与确保 x_i 位于间隔的正确一侧之间的关系

- 对于足够小的 ρ 值，如果输入数据是可以线性分类的，则软间隔支持向量机与硬间隔支持向量机将表现相同，但即使不可线性分类，软间隔支持向量机仍能学习出可行的分类规则



• 支持向量机

➤ 软间隔支持向量机的解法

计算（软间隔）支持向量机分类器一般采用简化方式，化为求解凸二次规划的最优化问题

• 原型：最小化问题可以用改写为目标函数可微的约束优化问题

对所有 $i \in \{1, 2, \dots, n\}$ ，引入变量

$$\xi_i = \max(0, 1 - y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b))$$

其中， ξ_i 为满足 $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i$ 的**最小非负数**，则优化问题可以描述为

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{n} \sum_{i=1}^n \xi_i + \rho \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i \\ & \xi_i \geq 0, \quad \forall 1 \leq i \leq n \end{aligned}$$



• 支持向量机

➤ 软间隔支持向量机的解法

- 对偶型

通过拉格朗日乘子法可得到拉格朗日函数

$$L(\mathbf{w}, b, \alpha, \xi, \mu)$$

$$= \frac{1}{2} \|\mathbf{w}\|^2 + \rho \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - y_i (< \mathbf{w}, \mathbf{x}_i > + b)) - \sum_{i=1}^n \mu_i \xi_i$$

其中 $\alpha_i \geq 0, \mu_i \geq 0$ 是拉格朗日乘子，令 $L(\mathbf{w}, b, \alpha, \xi, \mu)$ 对 w_i, b, ξ_i 的偏导为零，可得

$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i,$$

$$0 = \sum_{i=1}^n \alpha_i y_i,$$

$$\rho = \alpha_i + \mu_i.$$



• 支持向量机

➤ 软间隔支持向量机的解法

则得到对偶问题的优化表达式

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j x_i^T x_j \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, i = 1, 2, \dots, m \end{aligned}$$

• 次梯度下降

支持向量机的次梯度下降算法直接用表达式

$$f(w, b) = \left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(w_i x_i + b)) \right] + \lambda \|w\|^2$$

f 是 w 与 b 的凸函数，可以使用传统的梯度下降方法或随机梯度下降，其中不是在函数梯度的方向上前进，而是在从函数的次梯度中选出的向量的方向上前进



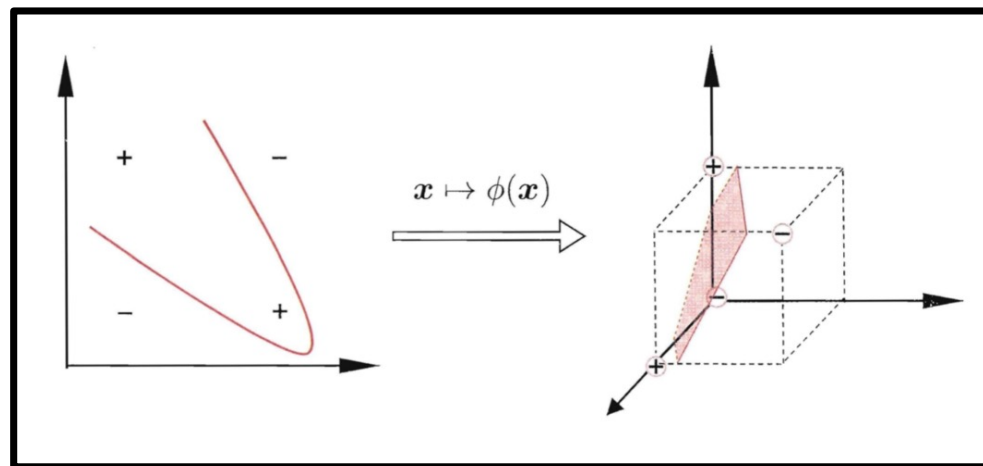
支持向量机原理

• 支持向量机

■ 非线性支持向量机的定义

通过核技巧应用于最大边界超平面来创建非线性分类器方法，把点换成非线性核函数，这就允许算法在变换后的特征空间中拟合最大间隔超平面，即令 $\phi(x)$ 表示将 x 映射后的特征向量，在特征空间中划分超平面所对应的模型可表示为

$$f(x) = \langle w, \phi(x) \rangle + b$$



- ① 如果给定数据样本为 $(0,0)$, $(1,0.8415)$, $(2,0.9093)$, $(3,0.1411)$, $(4,-0.7568)$, $(5,-0.9589)$, $(6,-0.2794)$, 分别用线性插值法、多项式插值法（拉格朗日插值法或牛顿法）、三次样条曲线插值法（边界条件为两次连续，二阶导数在终点为零），计算出相应的插值函数及误差余项
- ② 设有如下数据

x_i	0	0.25	0.5	0.75	1
y_i	1	1.2840	1.6487	2.1170	2.7183

利用最小乘法求该组数据的线性二次多项式拟合曲线

- ③ 在 $[-1,1]$ 上，求函数 $f(x) = |x|$ 在 $\mathcal{P}_3 = \text{span}\{1, x, x^3\}$ 中的最佳平方逼近函数



欢迎大家交流和讨论!

刘昌鑫

E-mail: changxinl@ecust.edu.cn